



BLM 2425 ALGORİTMA ANALİZİ

ÖDEV 2

Böl ve Yönet Algoritmaları

Öğrenci Adı: Sinem SARAĞ

Öğrenci Numarası: 22011647

Ders Grubu: 1

Dersin Eğitmeni: M. Elif KARSLIĞİL

Sunum Video Linki: <https://www.youtube.com/watch?v=F7LmdElo95w>

İçindekiler

Problemin Çözümü.....	3
1- Rastgele Dizilerin Üretimi.....	3
2- K-Way Merge Sort Algoritması	3
3- Çalışma Süresinin İncelenmesi ve Yorumlanması	4
Karşılaşılan Sorunlar	5
1- Rastgele Eleman Dizisi Oluşturulurken Yaşanan Bekleme Sorunu.....	5
2- K Adet Dizin Birleştirilmesi Sırasında Yaşanan İndisleme Problemi.....	5
Karmaşıklık Analizi	6
Ekran Çıktıları	7
N = 10 İçin Ekran Çıktısı	7
N = 100 İçin Ekran Çıktıları.....	7
N = 1.000 İçin Ekran Çıktıları.....	9
N = 10.000 İçin Ekran Çıktıları.....	10
N = 100.000 İçin Ekran Çıktıları.....	11
N = 1.000.000 İçin Ekran Çıktıları.....	13
N = 10.000.000 İçin Ekran Çıktıları.....	14

Problemin Çözümü

Bu problemde, k-way Merge Sort algoritmasının kullanılması istenmiştir. Problemin çözümü bu başlık altında iki kısım halinde anlatılacaktır: rastgele dizilerin üretilmesi ve k-way Merge Sort algoritmasıyla sıralanması.

Ayrıca, N ve k değerleri değiştirilerek elde edilen çalışma sürelerinin karşılaştırmalı grafiği ve bu grafil üzerinden yapılacak olan yorumlar da bu başlık içerisinde bir alt başlıkta incelenecektir.

1- Rastgele Dizilerin Üretimi

Problemin ilk kısmında, verilen eleman sayısına göre rastgele bir dizi oluşturulması istenmiştir. Bu amaçla, program girilen eleman sayısı (N) için birden başlayarak ardışık bir dizi yaratır. Örneğin, N = 10 için 1'den 10'a kadar sıralı [1,2,3,4,5,6,7,8,9,10] dizisi oluşturulur. Sonrasında bu dizinin elemanlarının, indisleri rand() fonksiyonu kullanılarak belirlenmiş rastgele değerler ile swap fonksiyonu kullanılarak yer değiştirilmesi ile dizi rastgele karıştırılır. Bu sayede, her elemanın eşsiz olduğu, karışık bir dizi elde edilir. Bu yöntem kullanılarak, büyük N değerleri için de benzersiz elemanlar içeren bir dizi üretilmiş olur. *Karşılaşılan Sorunlar* başlığı altında farklı bir yaklaşım daha ele alınarak bu yaklaşımın neden büyük en değerleri için daha iyi olduğu açıklanacaktır.

2- K-Way Merge Sort Algoritması

Problemin ikinci kısmında, oluşturulan rastgele dizinin k-way Merge Sort algoritmasıyla sıralanması gerekmektedir. Geleneksel Merge Sort algoritması, bir diziyi iki parçaya bölüp sıralarken, k-way Merge Sort algoritması diziyi k parçaya bölerek sıralamaktadır. Algoritmanın adımları şu şekildedir:

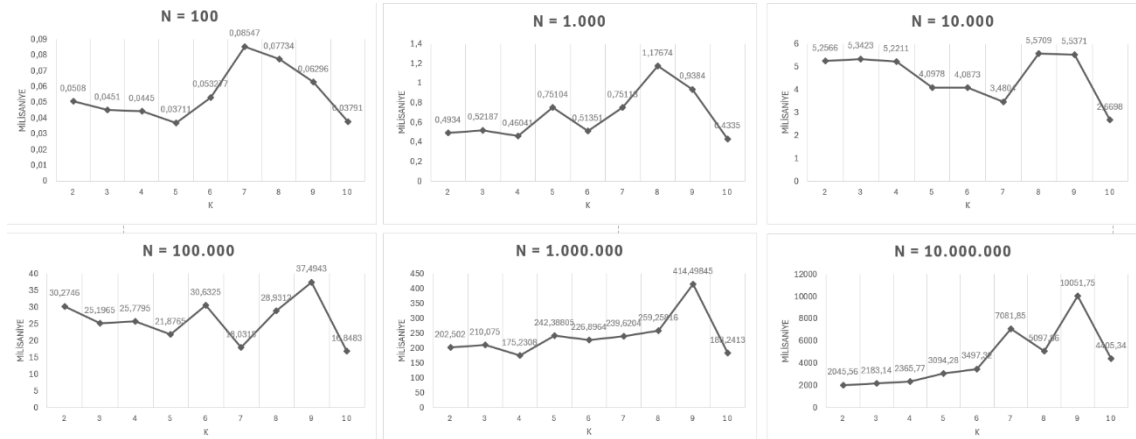
- 1) ***Dizinin Bölünmesi:*** İlk olarak, verilen diziyi k adet alt parçaya bölmek için dizinin uzunluğu k sayısına bölünerek her bir parçanın boyutu belirlenir. Eğer bölme işlemi tam bölünmezse (örneğin, dizinin uzunluğu k'nın tam katı değilse), ilk birkaç parçaya fazladan birer eleman eklenerek tüm elemanların kapsanması sağlanır.
- 2) ***Her Parçanın Sıralanması:*** Kademeli olarak alt parçalara bölünen dizinin her bir alt parçası, k-way Merge Sort algoritması kullanılarak kendi içinde sıralanır. Bu adım, algoritmanın böl ve yönet mantığına uygun olarak rekürsif bir yapıda gerçekleştirilir. Alt parçalar tek eleman uzunluğunda oluncaya kadar bölünme işlemi devam eder.
- 3) ***K Parçanın Birleştirilmesi:*** Kademeli olarak sıralanan alt parçalar, uzunluğu bir olan parçalardan başlanarak, merge fonksiyonu ile ikişerli olarak birleştirilir. İlk iki alt parça birleştirildikten sonra, sıradaki alt parça ile elde edilen sıralı dizi tekrar birleştirilir. Bu süreç, tüm alt parçalar birleştirilene kadar devam eder ve sonunda tüm dizi sıralı bir hale getirilir.

Bu algoritma, k parçaya bölmesinden kaynaklı geleneksel Merge Sort'a göre daha fazla bölme adımı ekler ve sıralamayı bu k parametresine göre yapar. k sayısının değerine göre bölme ve kıyaslama işlemlerinin sayıları artabilir. Bu durumun çalışma süresine etkisi bir sonraki bölümde grafikler üzerinden yorumlanacaktır.

3- Çalışma Süresinin İncelenmesi ve Yorumlanması

Algoritmanın çalışma süresini ölçmek için program; ödev dosyasında verilen eleman değerleri kullanılarak (N) çalıştırılmıştır. Program her N değeri için uygun diziyi oluşturduktan sonra; k değişkeninin 2'den 10'a kadar olan tüm değerleri ile sıralama işlemini gerçekleştirerek her bir sıralama işlemi için geçen süreyi ayrı ayrı ölçerek sıralama sonunda ekrana yazdırmaktadır. Her bir N değeri için bu program on defa çalıştırılarak, sıralama sürelerinin ortalaması hesaplanmıştır. Hesaplama işleminde kullanılan çıktıların ekran görüntüleri *Ekran Çıktıları* başlığı altına eklenecektir. Bu şekilde ortalama bir süre elde edilmesi hedeflenmiştir. Bu ortalamaların değişen k ve N değerlerine göre aldıkları değerler tablosu ve her bir k ve N değeri için çalışma süresinin karşılaştırıldığı grafik aşağıda gösterilmiştir¹:

k \ N	100	1000	10.000	100.000	1.000.000	10.000.000
2	0,0508	0,4934	5,2566	30,2746	202,502	2045,56
3	0,0451	0,52187	5,3423	25,1965	210,075	2183,14
4	0,0445	0,46041	5,2211	25,7795	175,2308	2365,77
5	0,03711	0,75104	4,0978	21,8765	242,38805	3094,28
6	0,053277	0,51351	4,0873	30,6325	226,8964	3497,32
7	0,08547	0,75118	3,4804	18,0318	239,6204	7081,85
8	0,07734	1,17674	5,5709	28,9312	259,25816	5097,86
9	0,06296	0,9384	5,5371	37,4943	414,49845	10051,75
10	0,03791	0,4335	2,6698	16,8483	183,2413	4405,34



N'in farklı değerleri için k değerinin grafikler üzerinde gösterilmesi

Tablo ve grafik incelendiğinde, N değeri arttıkça çalışma süresinin ani bir şekilde arttığı gözlemlenmektedir. Bu durum, sıralama işleminin büyüklükle doğru orantılı olarak daha fazla işlem gerektirdiğini göstermektedir. K-Way Merge Sort algoritması dizinin eleman sayısı büyüdükçe, daha fazla alt diziye ayırma, kıyaslama ve swap işlemi yapmaktadır. Bu nedenle çalışma süresi önemli ölçüde uzamaktadır.

k değeri, dizinin kaç parçaya bölüneceğini belirlediğinden ötürü çalışma süresini doğrudan etkilemektedir. Ancak k değerinin artması veya azalması, sıralama ve birleştirme işlemlerinin sayısını ve karmaşıklığını farklı şekillerde etkilediğinden aralarında k değeri ile hız arasında doğrusal bir ilişki kurulamamıştır. Nitekim gözlemler sonucu elde edilen değerler de doğrusal bir artış göstermemektedir.

k değeri arttıkça dizi daha fazla parçaya bölünmekte ve bu, her bir parçanın kendi arasında daha küçük boyutlarda sıralanmasını sağlamaktadır. Küçük parçalar üzerinde sıralama işleminin daha hızlı olması sebebiyle sıralama işleminin daha hızlı gerçekleşeceği söylenebilir. Ancak k değeri arttıkça, daha fazla rekürsif çağrı yapılmasının yanı sıra daha fazla sayıda parça olduğundan bu parçaların tekrar birleştirilmesi de, k değişkeninin küçük değerlerine göre, sürenin fazla olmasına sebep

1: Çalışma süresinin N değerine bağlı olarak çok hızlı değişmesi nedeniyle; tüm N değerleri için, k değerlerini tek bir çizgi grafiğinde göstermek yerine, her bir N değeri için bağımsız grafikler kullanılmıştır. Bu sayede k değerinin etkileri daha net incelenebilmiştir.

olacaktır. Özellikle büyük N değerlerinde k yüksek olduğunda, parçaların birleştirilmesinin fazla zaman alacağı söylenebilir. $N = 1.000.000$ ve $N = 10.000.000$ için $k=9$ için önemli miktarda performans düşüşü gözlenmesi bu nedenle açıklanabilir.

k değeri azaldıkça, dizi daha az sayıda parçaya bölünür. Bu durumda bölme sırasında yapılan rekürsif çağrılar ve birleştirme işlemleri daha az sayıda gerçekleşecek ve bu durum birleştirme süresini azaltacaktır. Ancak k değeri çok düşük olduğunda, diziden bölünen her parçanın boyutunun daha büyük olması gerektiğinden; bu parçaların sıralanması, k 'nin büyük değerleri kullanılarak oluşturulan parçalarla kıyaslandığında daha uzun sürecektir. Bu nedenle k değeri fazla küçük olduğunda da performans optimal seviyede olmayabilir.

Sonuç olarak, k değerinin artması küçük parçalar üzerinde daha hızlı sıralama işlemleri yapılmasını sağlarken, aşırı k değerlerinde birleştirme işlemleri sürenin artmasına neden olmaktadır. Küçük N değerlerinde, k değerinin küçük olduğu durumlarda (örn., $k = 2$, $k = 3$ veya $k=4$) algoritma, büyük k değerlerinden daha iyi performans göstermektedir. Küçük veri kümelerinde daha az parça ile işlem yapmak sıralama ve birleştirme işlemlerini daha optimize olmasını sağlamaktadır. k değerinin dizinin eleman sayısına göre, sıralama işlemi için yeterince küçük parçalara sahip olmak ve bununla birlikte parçalara ayırma işlemi için gereken rekürsif çağrı sayısını optimal seviyede tutmak aynı zamanda da birleştirme işlemlerini de yönetilebilir bir seviyede tutmak için eleman sayısına göre dikkatlice seçilmesi gerekir. k değeri çok yüksek olduğunda, birleştirme işlemlerinin karmaşıklığı arttığından performans düşüşü yaşandığı ve çok düşük k değerlerinde ise her bir parçanın sıralama süresi uzadığından ötürü sıralama süresinin arttığı gözlenmiştir.

Karşılaşılan Sorunlar

1- Rastgele Eleman Dizisi Oluşturulurken Yaşanan Bekleme Sorunu

Rastgele bir dizi oluşturmak için denenen ilk yaklaşım, `rand()` fonksiyonunu kullanarak her bir eleman için eşsiz değerler seçmektir. Seçilen her eleman `isExist` dizisinde ilgili indis değeri 0'dan 1'e güncellenecek şekilde tutulacak ve `rand()` tarafından sağlanan her elemanın bu dizideki karşılığı kontrol edilerek diziye atanması sağlanacaktı. Ancak, özellikle yüksek N değerlerinde bu yöntemin uzun süre beklemeye sebep olduğu gözlemlendi. Bunun sebebinin, N 'e yaklaştıkça; programın dizinin son elemanını tespit etmesinin ,zaten birçok sayının seçilmiş olmasından kaynaklı, zorlaşması olduğu anlaşıldı. Ödev kodunun içerisinde bu yaklaşıma dair kod kısmı yorum satırları içerisinde mevcuttur.

Bu sorunu aşmak için alternatif bir yöntem düşünüldü. İlk olarak N uzunluğunda sıralı bir dizi oluşturulup ardından bu dizinin elemanlarının; `rand()` kullanılarak elde edilen rastgele indisler ile yer değiştirilmesi yoluna gidildi. Bu yaklaşım, `rand()` ile tek tek eşsiz sayı bulma sürecini ortadan hız açısından avantaj sağladı. Böylece, özellikle yüksek N değerlerinde `rand()` fonksiyonunun bütün sayıları tespit etmesinin beklenmesine gerek olmaksızın dizi oluşturma işlemi tamamlandı.

2- K Adet Dizinin Birleştirilmesi Sırasında Yaşanan İndisleme Problemi

Bölme işlemi için düşünülmüş olan ilk yaklaşım, dizinin her bir parçası için farklı bir alan ayırarak gitmektir. Bunun için bir for döngüsü içerisinde her adımda; dizi elemanlarından, hesaplanan uzunluk değeri adedinde eleman başka bir diziye kopyalanarak bölme işlemleri gerçekleştirildi. Ancak daha sonra birleştirme aşamasında dizilerin uzunlukları her zaman birbirine eşit olmadığından bu yöntem erişim sorunlarına sebep oldu.

Daha sonra tekrardan tasarlanarak çözümde ise, yeni bir dizi açıp elemanları kopyalamak yerine; dizi içerisinde her adımda $k-1$ adet birbirine eş uzaklıklı noktanın (N , k 'ye tam bölünmediği zamanlar uzaklık, bölümden kalan değer kadar aralıkta bir fazla olarak hesaplanmaktadır) seçilmesi ile bölünme işlemi gerçekleştirildi.

Bölme işleminin yanı sıra diziyi birleştirme kısmında da k adet diziyi aynı anda birleştirirken seçilen noktaların aralarındaki mesafenin her zaman eşit olmamasından kaynaklı bu parçaların başlangıç ve bitiş indislerini doğru şekilde tespit etmek zorlaştı. Bu hatalı indis hesaplamaları zaman zaman erişim hatalarına (out of bounds) yol açtı, programın çökmesine sebep oldu veya bazı durumlarda sıralama işleminde bazı elemanların yerlerine konulamamasına sebep oldu. Bu sorunu çözmek için ise k adet dizinin içerisinde sırayla seçilen diziler ikiye ikiye birleştirilerek ilerlendi. Bu şekilde dizi parçalarını tutan indislerin yönetimi daha kolay hale getirilerek dizi erişim hataları ve dizideki sıralama hatalarının önüne geçilmiş oldu.

Karmaşıklık Analizi

```
FUNCTION kWayMergeSort(arr, left, right, k)
  IF left >= right THEN
    RETURN

  segmentSize = (right - left + 1) / k
  extras = (right - left + 1) % k
  start = left
  midPoints = array of size k

  FOR i = 0 TO k - 1
    end = start + segmentSize - 1
    IF i < extras THEN
      end += 1

    midPoints[i] = end
    CALL kWayMergeSort(arr, start, end, k)

    start = end + 1
  ENDFOR

  FOR i = 1 TO k - 1
    CALL merge(arr, left, midPoints[i - 1], midPoints[i])
  ENDFOR
ENDFUNCTION
```

Yanda, algoritmanın en fazla işlem maliyetine sahip olan kWayMergeSort fonksiyonunun sözde kodu yer almaktadır. Bu fonksiyon içerisinde karmaşık kısım k parçanın birleştirildiği merge fonksiyonudur. Çünkü bu işlem her rekürsif adımda N eleman üzerinden yapılmaktadır ve bu da zaman karmaşıklığının artmasında etkili olmaktadır.

kWayMergeSort fonksiyonu içerisinde, $left < right$ şartını sağlayan diziyi her adımda k parçaya ayrılır. Her parça kendi içinde rekürsif olarak sıralanır ve ardından merge fonksiyonu ile birleştirilir.

kWayMergeSort fonksiyonu, diziyi her seferinde k parçaya böler. Bu durumda N boyutunda bir diziyi k parçaya böldüğümüzde her parça yaklaşık olarak N/k uzunluğunda olur. Bu bölme işlemi her adımda tekrarlandığı için dizinin tamamı sıralanana kadar logaritmik olarak derinleşen bir çağrı yapısı oluşur. Böylece, rekürsif çağrılarının derinliği $\log_k(N)$ olarak hesaplanır.

merge işlemi, k parçayı birleştirirken yaklaşık olarak N eleman üzerinden işlem yapar. Bu birleştirme işlemi her rekürsif çağrı düzeyinde yapılır ve her seferinde N eleman üzerinden çalışır. Dolayısıyla, birleştirme işlemi her çağrı düzeyi için $O(N)$ zaman karmaşıklığına sahiptir. Bu durumda kwayMergeSort algoritması, her derinlikte $O(N)$ birleştirme işlemi yapar ve toplamda $\log_k(N)$ derinliğe sahiptir. Bu bilgiler toplanarak genel zaman karmaşıklığı hesaplandığında toplam karmaşıklık $O(N * \log_k(N))$ olarak bulunur.

$$T(N) = O(N * \log_k(N))$$

Ekran Çıktıları

N = 10 İçin Ekran Çıktısı

```
Welcome, please enter the number of elements: 10
Original array created:
8 2 5 10 3 6 7 1 9 4

Original array restored. Starting timer now for k = 2
Sorting completed, timer is stopping now.
Sorting took for k = 2: 0.013900 milliseconds.

Original array restored. Starting timer now for k = 3
Sorting completed, timer is stopping now.
Sorting took for k = 3: 0.004200 milliseconds.

Original array restored. Starting timer now for k = 4
Sorting completed, timer is stopping now.
Sorting took for k = 4: 0.004200 milliseconds.

Original array restored. Starting timer now for k = 5
Sorting completed, timer is stopping now.
Sorting took for k = 5: 0.005000 milliseconds.

Original array restored. Starting timer now for k = 6
Sorting completed, timer is stopping now.
Sorting took for k = 6: 0.005400 milliseconds.

Original array restored. Starting timer now for k = 7
Sorting completed, timer is stopping now.
Sorting took for k = 7: 0.011100 milliseconds.

Original array restored. Starting timer now for k = 8
Sorting completed, timer is stopping now.
Sorting took for k = 8: 0.008500 milliseconds.

Original array restored. Starting timer now for k = 9
Sorting completed, timer is stopping now.
Sorting took for k = 9: 0.007600 milliseconds.

Original array restored. Starting timer now for k = 10
Sorting completed, timer is stopping now.
Sorting took for k = 10: 0.006200 milliseconds.

After sort process:
1 2 3 4 5 6 7 8 9 10
```

Dizinin doğru sıralandığından emin olunması adına program diziyi sıralamadan önce ve sonra diziyi ekrana yazdırmaktadır. N değişkeninin, ödev dokümanında verilen değerler için oluşturulan dizinin sıralanmasının kontrolü zor olacağından sıralama işlemi N = 10 ile denenmiştir.

N = 100 İçin Ekran Çıktıları

```
Welcome, please enter the number of elements: 100

Original array restored. Starting timer now for k = 2
Sorting completed, timer is stopping now.
Sorting took for k = 2: 0.060400 milliseconds.

Original array restored. Starting timer now for k = 3
Sorting completed, timer is stopping now.
Sorting took for k = 3: 0.094900 milliseconds.

Original array restored. Starting timer now for k = 4
Sorting completed, timer is stopping now.
Sorting took for k = 4: 0.070700 milliseconds.

Original array restored. Starting timer now for k = 5
Sorting completed, timer is stopping now.
Sorting took for k = 5: 0.053500 milliseconds.

Original array restored. Starting timer now for k = 6
Sorting completed, timer is stopping now.
Sorting took for k = 6: 0.079700 milliseconds.

Original array restored. Starting timer now for k = 7
Sorting completed, timer is stopping now.
Sorting took for k = 7: 0.126100 milliseconds.

Original array restored. Starting timer now for k = 8
Sorting completed, timer is stopping now.
Sorting took for k = 8: 0.116100 milliseconds.

Original array restored. Starting timer now for k = 9
Sorting completed, timer is stopping now.
Sorting took for k = 9: 0.091700 milliseconds.

Original array restored. Starting timer now for k = 10
Sorting completed, timer is stopping now.
Sorting took for k = 10: 0.051400 milliseconds.
```

```
Welcome, please enter the number of elements: 100

Original array restored. Starting timer now for k = 2
Sorting completed, timer is stopping now.
Sorting took for k = 2: 0.045800 milliseconds.

Original array restored. Starting timer now for k = 3
Sorting completed, timer is stopping now.
Sorting took for k = 3: 0.038000 milliseconds.

Original array restored. Starting timer now for k = 4
Sorting completed, timer is stopping now.
Sorting took for k = 4: 0.039700 milliseconds.

Original array restored. Starting timer now for k = 5
Sorting completed, timer is stopping now.
Sorting took for k = 5: 0.031800 milliseconds.

Original array restored. Starting timer now for k = 6
Sorting completed, timer is stopping now.
Sorting took for k = 6: 0.045500 milliseconds.

Original array restored. Starting timer now for k = 7
Sorting completed, timer is stopping now.
Sorting took for k = 7: 0.070100 milliseconds.

Original array restored. Starting timer now for k = 8
Sorting completed, timer is stopping now.
Sorting took for k = 8: 0.068700 milliseconds.

Original array restored. Starting timer now for k = 9
Sorting completed, timer is stopping now.
Sorting took for k = 9: 0.058000 milliseconds.

Original array restored. Starting timer now for k = 10
Sorting completed, timer is stopping now.
Sorting took for k = 10: 0.035600 milliseconds.
```



```
Welcome, please enter the number of elements: 1000

Original array restored. Starting timer now for k = 2
Sorting completed, timer is stopping now.
Sorting took for k = 2: 0.602800 milliseconds.
-----

Original array restored. Starting timer now for k = 3
Sorting completed, timer is stopping now.
Sorting took for k = 3: 0.561990 milliseconds.
-----

Original array restored. Starting timer now for k = 4
Sorting completed, timer is stopping now.
Sorting took for k = 4: 0.466980 milliseconds.
-----

Original array restored. Starting timer now for k = 5
Sorting completed, timer is stopping now.
Sorting took for k = 5: 0.772100 milliseconds.
-----

Original array restored. Starting timer now for k = 6
Sorting completed, timer is stopping now.
Sorting took for k = 6: 0.562800 milliseconds.
-----

Original array restored. Starting timer now for k = 7
Sorting completed, timer is stopping now.
Sorting took for k = 7: 0.853600 milliseconds.
-----

Original array restored. Starting timer now for k = 8
Sorting completed, timer is stopping now.
Sorting took for k = 8: 1.262900 milliseconds.
-----

Original array restored. Starting timer now for k = 9
Sorting completed, timer is stopping now.
Sorting took for k = 9: 0.978800 milliseconds.
-----

Original array restored. Starting timer now for k = 10
Sorting completed, timer is stopping now.
Sorting took for k = 10: 0.589160 milliseconds.
```

```

Welcome, please enter the number of elements: 10000

Original array restored. Starting timer now for k = 2
Sorting completed, timer is stopping now.
Sorting took for k = 2: 5.990100 milliseconds.

-----

Original array restored. Starting timer now for k = 3
Sorting completed, timer is stopping now.
Sorting took for k = 3: 5.473600 milliseconds.

-----

Original array restored. Starting timer now for k = 4
Sorting completed, timer is stopping now.
Sorting took for k = 4: 6.966900 milliseconds.

-----

Original array restored. Starting timer now for k = 5
Sorting completed, timer is stopping now.
Sorting took for k = 5: 6.428700 milliseconds.

-----

Original array restored. Starting timer now for k = 6
Sorting completed, timer is stopping now.
Sorting took for k = 6: 5.752800 milliseconds.

-----

Original array restored. Starting timer now for k = 7
Sorting completed, timer is stopping now.
Sorting took for k = 7: 2.237200 milliseconds.

-----

Original array restored. Starting timer now for k = 8
Sorting completed, timer is stopping now.
Sorting took for k = 8: 5.693500 milliseconds.

-----

Original array restored. Starting timer now for k = 9
Sorting completed, timer is stopping now.
Sorting took for k = 9: 5.696500 milliseconds.

-----

Original array restored. Starting timer now for k = 10
Sorting completed, timer is stopping now.
Sorting took for k = 10: 3.828000 milliseconds.

```

```
Welcome, please enter the number of elements: 10000

Original array restored. Starting timer now for k = 2
Sorting completed, timer is stopping now.
Sorting took for k = 2: 3.978109 milliseconds.

-----

Original array restored. Starting timer now for k = 3
Sorting completed, timer is stopping now.
Sorting took for k = 3: 4.943780 milliseconds.

-----

Original array restored. Starting timer now for k = 4
Sorting completed, timer is stopping now.
Sorting took for k = 4: 6.706208 milliseconds.

-----

Original array restored. Starting timer now for k = 5
Sorting completed, timer is stopping now.
Sorting took for k = 5: 3.989990 milliseconds.

-----

Original array restored. Starting timer now for k = 6
Sorting completed, timer is stopping now.
Sorting took for k = 6: 3.856208 milliseconds.

-----

Original array restored. Starting timer now for k = 7
Sorting completed, timer is stopping now.
Sorting took for k = 7: 3.308690 milliseconds.

-----

Original array restored. Starting timer now for k = 8
Sorting completed, timer is stopping now.
Sorting took for k = 8: 5.612080 milliseconds.

-----

Original array restored. Starting timer now for k = 9
Sorting completed, timer is stopping now.
Sorting took for k = 9: 5.991500 milliseconds.

-----

Original array restored. Starting timer now for k = 10
Sorting completed, timer is stopping now.
Sorting took for k = 10: 2.471090 milliseconds.
```



```
Original array restored. Starting timer now for k = 10
Sorting completed, timer is stopping now.
Sorting took for k = 10: 5657.530400 milliseconds.
```

```
Original array restored. Starting timer now for k = 10
Sorting completed, timer is stopping now.
Sorting took for k = 10: 5464.216700 milliseconds.
```

```
Original array restored. Starting timer now for k = 10
Sorting completed, timer is stopping now.
Sorting took for k = 10: 5429.318800 milliseconds.
```

```
Original array restored. Starting timer now for k = 10
Sorting completed, timer is stopping now.
Sorting took for k = 10: 5445.124300 milliseconds.
```